# Project Report-III

## *Parking Lot Management*

### Problem statement:

To build a “Parking Lot Management” system on a web-application that tracks entry and exit of vehicles in a parking lot, track available parking spaces, and calculate parking fees.

### Aim:

To build a user-friendly, easily understandable, yet detailed parking lot management web-application using recently learnt modules of Figma (Prototype), Angular (Frontend), C# (API) and SQL (Backend).

The parking lot has a fixed number of slots (e.g., 50).

Users:

- Parking Attendant – records vehicle entry and exit.
- Supervisor – monitors parking usage and daily reports.

### Introduction:

Following the second report submitted earlier in this report I have covered the project update which features the “Future improvements” from which I have implemented a few methods. This report contains the following:

1. **IOT mock page linking.**
2. **ML based number plate detection system.**
3. Addition of splash page
4. Redesigned supervisor page.

## IOT mock page:


The addition of IOT mock page which can be later linked with a real IOT module has been added to the project as a separate angular page that communicates directly with the API. For now, the physical presence mocking is done by manually toggling the slots by visiting the webpage.

- **Free**
- **Entry Waiting**
- **Exit Waiting**
- **Occupied**

Free:

When a slot has not been booked and when no physical presence of vehicle is detected the slot is marked as “FREE”.

This slot is available for booking where the backend says the slot is free from reservation, blockage or occupied.

This is the only phase where booking will be available for the current slot.

Entry waiting:

When a free slot has been booked by the attendant during entry and the IOT signals waiting the slot turns showing a yellow box in the page with “ENTRY WAITING”.

This phase is showed when the slot has been booked in backend but the physicals presence of vehicle is yet to be detected. Once a physical presence of vehicle is detected by the IOT the slot falls into “OCCUPIED” phase.

No duplicate or overlapped booking can be done on this slot once booked.

Exit waiting:

This phase of the slot is when the driver leaves a slot but is not officially exited the parking lot yet. The IOT signals a physical absence when the backend signals a presence of vehicle inside the parking lot because the vehicle hasn’t exited the parking lot yet.

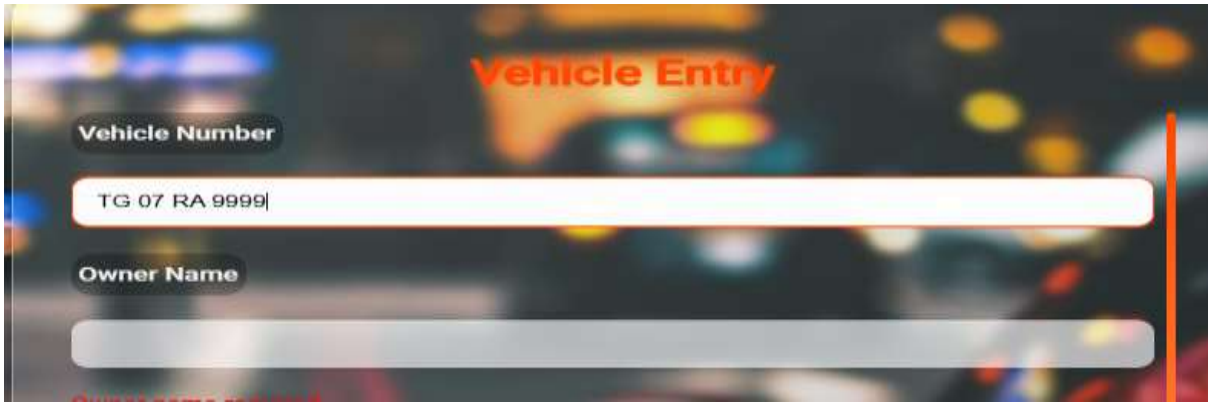
This is exact opposite in all way of ‘Entry waiting’ slot properties. This phase is shown with “EXIT WAITING” in the mock IOT page. The slot cannot be booked yet.

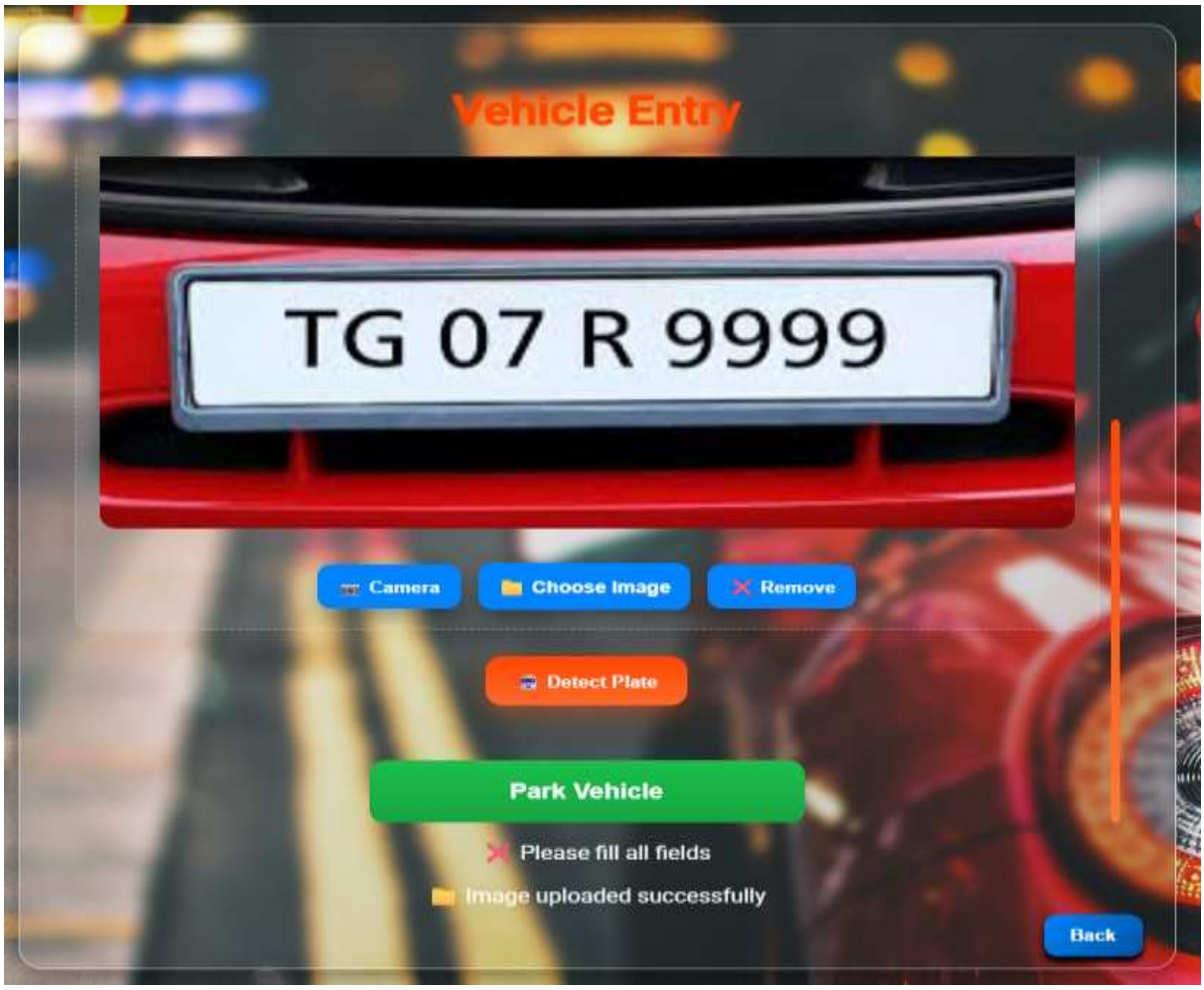
Occupied:

This is the actual phase after slot booking where both the records and IOT signals a presence of vehicle. During this time no other vehicle slot can be booked on this slot.

When a vehicle physically exits, this slot automatically falls to “Entry waiting” phase, where the attendant needs to exit the vehicle to make the slot to “FREE” phase.

## ML number-plate detection:

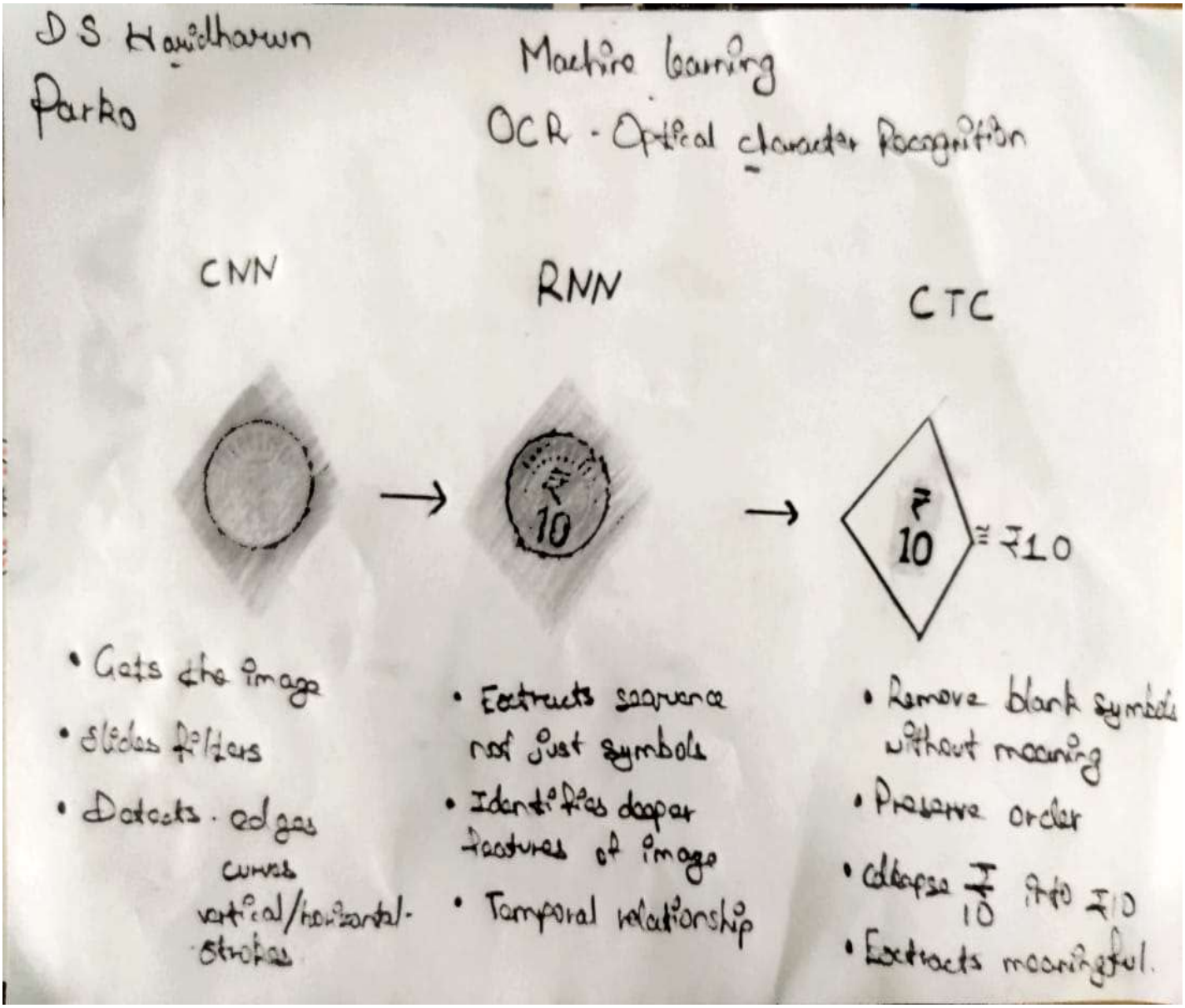





The number plate detection system is a brand new feature added to my parking lot project that gets input as an image file and detects the vehicle number from the number plate of vehicle and auto fills the ‘Vehicle number’ field.

This reduces manual entry time as well as covers the entire car in the photo will be added to the records for security. This allows both faster booking of slots as well as more security to the vehicles parked in the lot by storing the captured image along.

I used OCR – Optical Character Recognition a machine learning technique to detect the numbers / characters from the number plate of vehicle and fill the input.




The above image showcases the architecture flow OCR model I have used:

CNN – Convolutional Neutral Network:

- Gets the image or output from the image capturing system (camera or upload).
- Slides different filters like HSV, Grayscale etc.
- Helps detects edges, curves, vertical or horizontal strokes.

RNN – Recurrent Neural Network:

- Extracts sequence, not just plain symbols from the image.
- Identifies deeper and more clear features from the image.
- Forms temporal relationship and leaves the rest for next module.

CTC – Connectionist Temporal Classification:

- Removes all blank symbols and unwanted features from the extracted image. For example, here in the image its clearly shown that the commas (“ ”) over the coin are removed entirely leaving us with only a human understandable text and symbols.
- Preserves an order. Top to Bottom or Left to Right is used as an usual ordering method.
- Collapses the identified order into a straight sequence for feeding into extraction.
- Finally extracts the meaningful text from the image. (₹10).

## Splash page:


Page loading might take some time to load and render, especially when there is more component usage in the webpage. This results in the user seeing unloaded and incomplete page for a while until the page renders. To prevent this and improve user

experience I have added a splash page that automatically plays when a page starts to load or render its components.

For the splash screen I used Cap-cut video editing tool to create a 2 seconds long page with simple animations and imported it into the project.

The splash page especially contains the project app logo, name and the ‘Loading…’ text to let user know that the page is loading.

## Supervisor page modification:

Earlier the supervisor page had 3 main components, The income chart, the live records table and the slot status bar. But, since we already have the slot status bar in the attendant page, I have redesigned it and added a total vehicle count chart.

It showcases the number of vehicles parked on a day according to the filters applied in a bar chart. The peak/ highest parking count is marked as the peak-day and shown in a different colour than the rest.